



Deakin University

Project Title: Smart Home Lighting System

Final Project Report

Student Name: Hayes Duong

Student ID: 222610226

Date of Submission: Oct 2nd, 2025

I. Introduction

Modern smart-home lighting products often deliver basic remote control but struggle with real-world needs such as easy onboarding, cross-vendor interoperability, and—critically—scalability when the number of devices, events, and automations grows. This project addresses those gaps by designing, building, and deploying a cloud-backed, event-driven Smart Home Lighting System that is both user-friendly and demonstrably scalable.

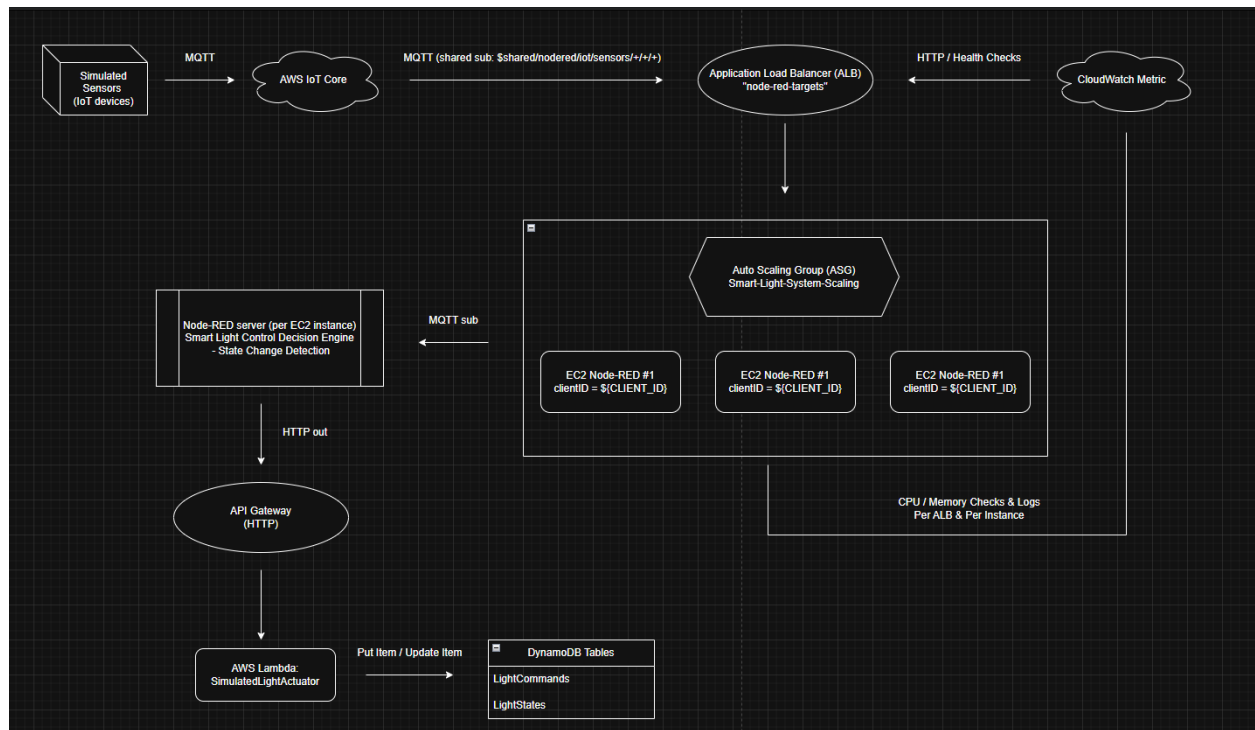
The solution couples low-cost IoT endpoints (simulated sensors/actuators for this submission) with a cloud control plane on AWS. Device telemetry is ingested via AWS IoT Core (MQTT), processed by an automation/decision layer built with Node-RED, and executed through microservices exposed over Amazon API Gateway and implemented with AWS Lambda. System state and command history persisted in Amazon DynamoDB using an append-only pattern (separate tables for LightCommands and LightStates) to improve traceability and reliability.

To satisfy scalability requirements, the design intentionally mixes serverless and auto-scale components:

- Event-driven microservices (API Gateway + Lambda + DynamoDB) scale automatically with request concurrency and require no server management.
- Long-lived, interactive automation tooling (Node-RED) runs on EC2 behind an Application Load Balancer (ALB) and an Auto Scaling Group (ASG).
- Health checks and CloudWatch metrics drive elastic scale-out/scale-in of Node-RED instances while keeping the service stateless at the compute layer.

Functionally, the system implements core lighting behaviors, e.g., turning lights on when motion is detected under low ambient lux, time-of-day brightness policies, and resilient actuator updates—while remaining deliberately simple so that the focus of this report is on the architecture, elasticity, and operational evidence rather than complex business logic.

II. System Design Overview



Picture 1 – Smart Home Lighting System diagram.

A – The architecture behind

The solution follows a hybrid event-driven + auto-scaling architecture (Picture 1):

- **IoT Ingestion Plane:** Simulated sensors publish telemetry via **MQTT** to **AWS IoT Core**.
- **Decision Plane:** **Node-RED** runs on **EC2** instances inside an **Auto Scaling Group (ASG)** behind an **Application Load Balancer (ALB)**. Node-RED subscribes to device topics and applies automation rules.
- **Actuation Plane:** Decisions are executed through a stateless HTTP endpoint exposed by **Amazon API Gateway** and implemented with **AWS Lambda** (SimulatedLightActuator).
- **Data Plane:** **Amazon DynamoDB** stores command history (LightCommands) and the latest append-only light states (LightStates).
- **Observability & Health:** **CloudWatch** collects metrics and drives scaling/health checks for the ALB/ASG stack and serverless components.

This separation (ingestion → decision → actuation → persistence) keeps services loosely coupled and independently scalable.

B – Components & Responsibilities

Simulated Sensors (Clients)

- Publish ambient light / motion readings over MQTT to AWS IoT Core topic
``iot/sensors/floor{floor}/room{room}/sensor_{id}``
- Use a shared subscription pattern for fan-out across Node-RED instance:
``$share/nodered/iot/sensors/+/+/+``

AWS IoT Core

- Managed MQTT broker and device registry.
- Routes telemetry to consumers (Node-RED); optional rules can fan out to additional sinks if required.

Node-RED (Decision Engine)

- Hosted per EC2 instance; each instance is stateless at compute layer.
- Subscribes to IoT topics; apply rules with state-change detection.
- Posts HTTP commands to API Gateway directly via URL: ``POST https://kfctm4ety1.execute-api.ap-southeast-2.amazonaws.com/lights/{payload.lightId}/state``

ALB + ASG (Control Plane Scalability)

- ALB distributes inbound HTTP (Node-RED UI), performs health checks.
- ASG scales Node-RED horizontally based on CloudWatch Metrics (%CPU Utilization; 60s instance warm-up)

API Gateway + Lambda (Simulated Light Actuator)

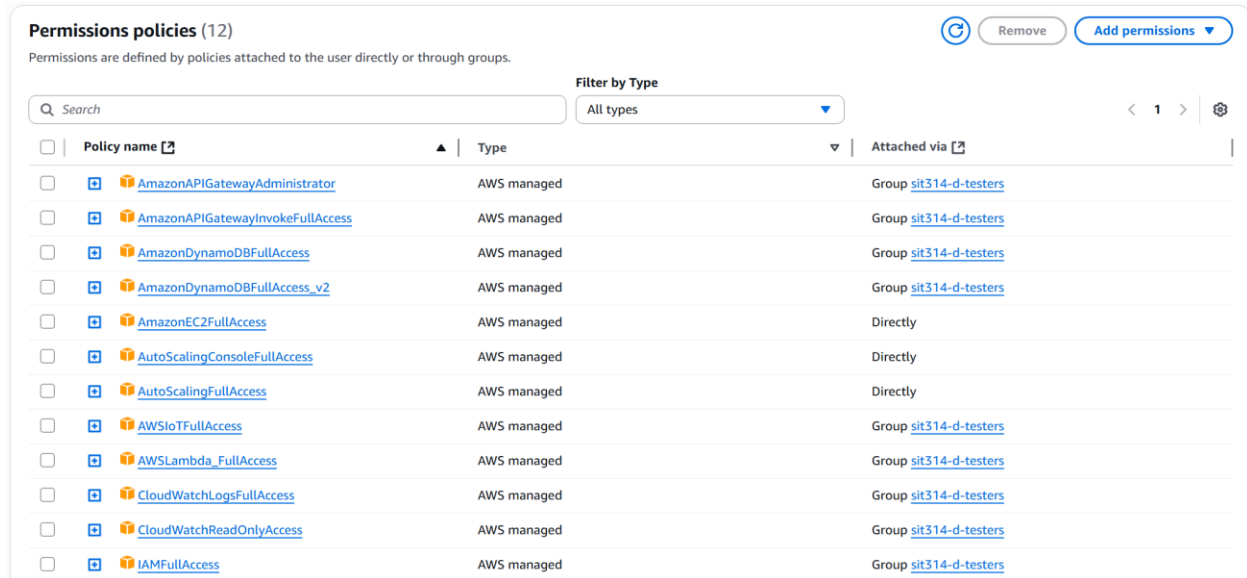
- API Gateway exposes a public HTTPS interface with request validation and throttling.
- AWS Lambda (SimulatedLightActuator) applies commands, logs durable intent/state, and returns the effective light state.

DynamoDB (Persistence)

- LightCommands: immutable audit log for each command
(commandId/timestamp/lightId/action/params/reason/status)

- LightStates: time-stamped state records per light.

C – Security considerations



Permissions policies (12)

Permissions are defined by policies attached to the user directly or through groups.

Filter by Type: All types

☐	Policy name	Type	Attached via
☐	AmazonAPIGatewayAdministrator	AWS managed	Group sit314-d-testers
☐	AmazonAPIGatewayInvokeFullAccess	AWS managed	Group sit314-d-testers
☐	AmazonDynamoDBFullAccess	AWS managed	Group sit314-d-testers
☐	AmazonDynamoDBFullAccess_v2	AWS managed	Group sit314-d-testers
☐	AmazonEC2FullAccess	AWS managed	Directly
☐	AutoScalingConsoleFullAccess	AWS managed	Directly
☐	AutoScalingFullAccess	AWS managed	Directly
☐	AWSIoTFullAccess	AWS managed	Group sit314-d-testers
☐	AWSLambda_FullAccess	AWS managed	Group sit314-d-testers
☐	CloudWatchLogsFullAccess	AWS managed	Group sit314-d-testers
☐	CloudWatchReadOnlyAccess	AWS managed	Group sit314-d-testers
☐	IAMFullAccess	AWS managed	Group sit314-d-testers

Picture 2 – Permitted Policies for IAM user.

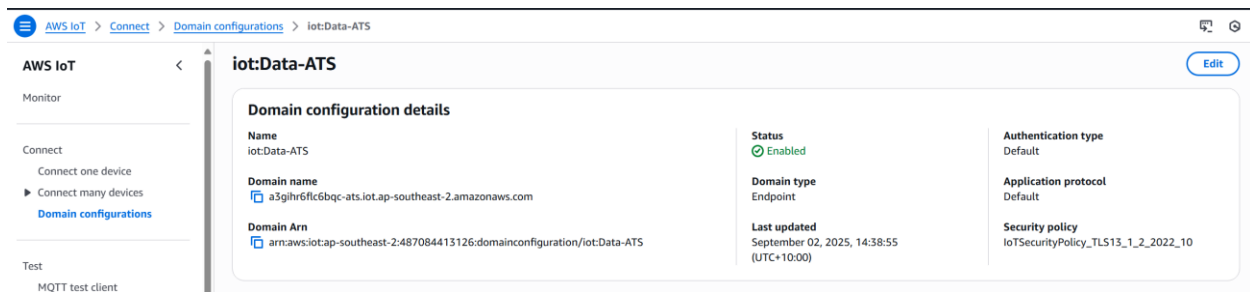
- All cloud work was performed using an IAM user (rather than root account). Picture 2 shows the set of policies attached during development to enable building and validating the end-to-end prototype (API Gateway, Lambda, DynamoDB, EC2/ASG/ALB, CloudWatch, IoT Core).

III. Implementation

A – AWS IoT, Lambda, API Gateway & DynamoDB Tables preparation

For AWS IoT Core, the following steps were followed:

1. Navigate to AWS IoT > Domain configuration
2. Choose “Create domain configuration” and named this configuration as “iot:Data-ATS” while leaving other options / fields as default.



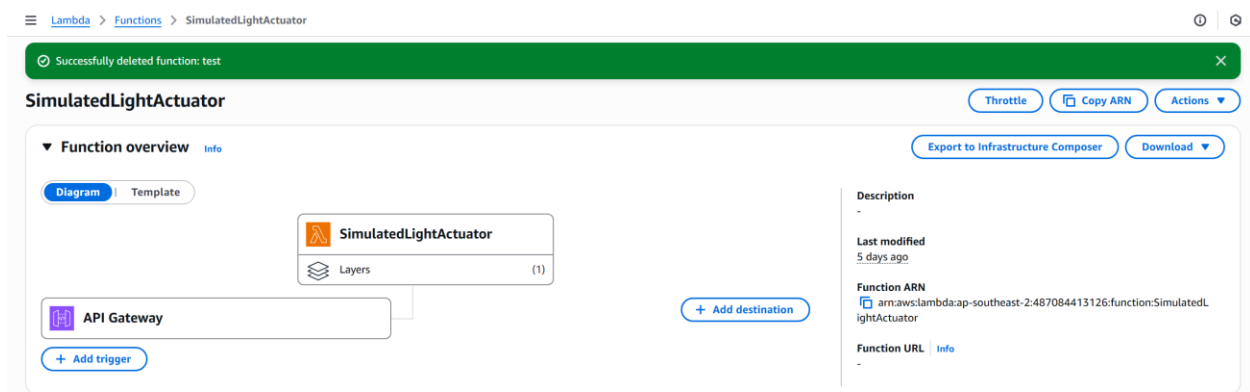
Picture 3 – “iot:Data-ATS” Domain configuration

For Lambda, the following steps were followed:

1. Navigate to Lambda > “Create function” > name “SimulatedLightActuator” and leave other options / fields as default.
2. The code for this function is found in “A. Lambda code for SimulatedLightActuator” of Appendix Section.

For API Gateway, the following steps were followed:

1. Navigate to API Gateway > “Create an API” > HTTP API “Build” button
2. Name: “SmartLighting-HTTP”; Integration: “Lambda” & choose “SimulatedLightActuator” from the list.
3. Route:
 - a. Method: “POST”
 - b. Resource path: “/lights/{lightId}/state”
4. Enable CORS



Picture 4 – “SimulatedLightActuator” Lambda function with “SmartLighting-HTTP” API Gateway as its trigger.

For DynamoDB Tables:

1. Navigate to DynamoDB > “Create table”

2. For “LightCommands” table:
 - a. Name: “LightCommands”
 - b. Partition key: “commandId”
 - c. Sort key: “timestamp”
3. For “LightStates” table:
 - a. Name: “LightStates”
 - b. Partition key: “lightId”
 - c. Sort key: “timestamp”

The screenshot shows the AWS DynamoDB console interface. On the left is a navigation menu with options like Dashboard, Tables, Explore items, PartiQL editor, Backups, and Exports to S3. The main area is titled 'Tables (3)' and contains a table listing the configured tables. The table has columns for Name, Status, Partition key, Sort key, Indexes, Replication Regions, Deletion protection, Favorite, and Read capacity mode. Two tables are listed: 'LightCommands' and 'LightStates', both with a status of 'Active'.

Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection	Favorite	Read capacity mode
LightCommands	Active	commandId (S)	timestamp (N)	0	0	Off	☆	On-demand
LightStates	Active	lightId (S)	timestamp (N)	0	0	Off	☆	On-demand

Picture 5 – “LightCommands” & “LightStates” DynamoDB tables.

B – AWS Amazon Machine Image (AMI) preparation

For this project, an “Amazon Linux 2023 kernel-6.1 AMI”, “t3.micro” is being used and the following steps were implemented to hosting a Node-RED on this EC2 instance:

1. Update system packages: `sudo yum update -y``
2. Install Node.js & verify the installation:


```

` curl -fsSL https://rpm.nodesource.com/setup_18.x | sudo bash -`
` sudo yum install -y nodejs`
` node -version`
      
```
3. Install Node-RED globally: `sudo npm install -g --unsafe-perm node-red``
4. Enable and start Node-RED service:

```

` sudo systemctl enable node-red`

` sudo systemctl start node-red`

```

As this project involved in the usage of ASG, thus, each Node-RED instance requires a unique MQTT clientId because AWS IoT Core enforces one active connection per clientId.

When multiple instances share the same clientId, only the most recently connected instance remains active while others are automatically disconnected.

Since this system uses both ALB and ASG, a shared subscription (\$share/nodered/iot/sensors/+/+/) is being used to distribute sensor messages across multiple Node-RED instances for horizontal scaling, each instance must maintain its own independent MQTT connection.

Without unique clientIds, the auto-scaling architecture would fail because only one instance could process sensor data, defeating the purpose of load distribution.

For this reason, the following steps were implemented this by deriving each instance's clientId from its EC2 instance-id (node-red-\${instance-id}), ensuring uniqueness across all ASG-launched instances.

1. Configure unique MQTT clientId

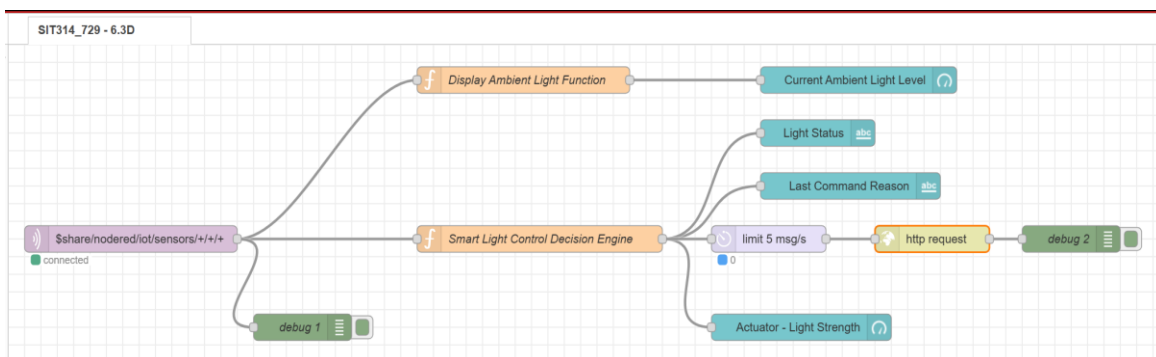
```
# Create environment file
sudo mkdir -p /etc/node-red
echo "CLIENT_ID=node-red-$(curl -s http://169.254.169.254/latest/meta-data/instance-id)" > /etc/node-red/env

# Create systemd override
sudo mkdir -p /etc/systemd/system/node-red.service.d
sudo tee /etc/systemd/system/node-red.service.d/override.conf > /dev/null <<'EOF'
[Service]
EnvironmentFile=/etc/node-red/env
EOF

# Reload and restart
sudo systemctl daemon-reload
sudo systemctl restart node-red
```

Picture 6 – Commands were used for configuring unique MQTT clientId.

2. Access Node-RED hosted by this EC2 instance by open browser to: `http://<ec2-public-ip>:1880`



Picture 7 – Node-RED configuration for this project

3. For this project, the Node-RED configuration is followed according to Picture 3, where the MQTT In (or MQTT-broker) node is set up as follows:

The image displays three screenshots of Node-RED node configuration interfaces:

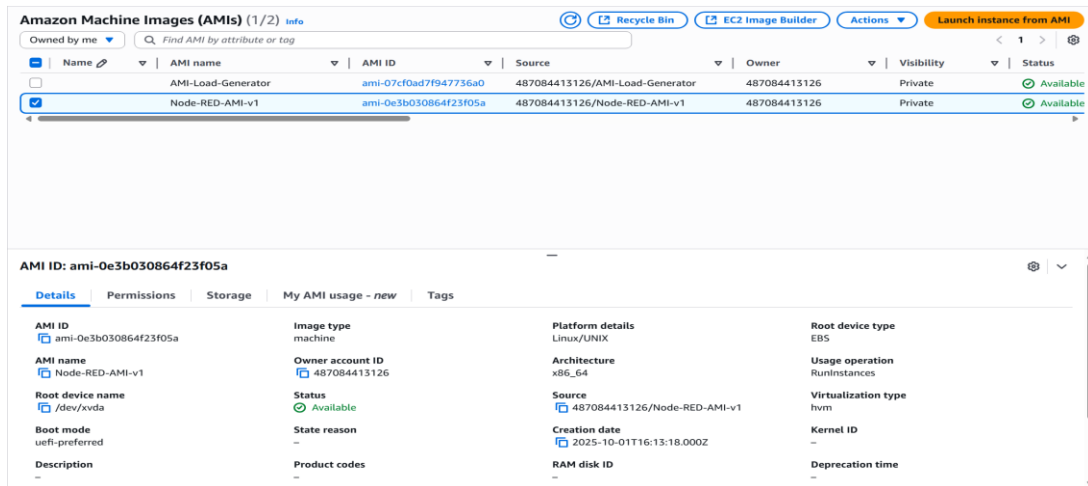
- MQTT-broker node:** The 'Edit mqtt-broker node' window shows the 'Properties' tab with 'Name' set to 'AWS IoT Core'. The 'Connection' tab shows 'Server' as 'a3gihrfic6bgc-ats.iot.ap-southeast-2.amazonaws.com', 'Port' as '8883', 'Connect automatically' checked, 'Use TLS' checked with 'light_sensor' selected, 'Protocol' as 'MQTT V3.1.1', 'Client ID' as '\$(CLIENT_ID)', 'Keep Alive' as '60', and 'Session' with 'Use clean session' checked.
- TLS-config node:** The 'Edit mqtt-broker node > Edit tls-config node' window shows the 'Properties' tab with 'Use key and certificates from local files' checked. Fields include 'Certificate' (/opt/node-red/certs/light_sensor.cert.pem), 'Private Key' (/opt/node-red/certs/light_sensor.private.key), 'Passphrase' (private key passphrase (optional)), 'CA Certificate' (/opt/node-red/certs/root-CA.crt), 'Verify server certificate' checked, 'Server Name' (for use with SNI), 'ALPN Protocol' (for use with ALPN), and 'Name' (light_sensor).
- HTTP request node:** The 'Edit http request node' window shows the 'Properties' tab with 'Method' as 'POST', 'URL' as 'https://kfcfm4ely1.execute-api.ap-southeast-2.amazonaws.com', 'Enable secure (SSL/TLS) connection' checked, 'Use authentication' checked, 'Enable connection keep-alive' checked, 'Use proxy' checked, 'Only send non-2xx responses to Catch node' checked, 'Disable strict HTTP parsing' checked, 'Return' as 'a UTF-8 string', and 'Headers' with 'Content-Type' set to 'application/json'.

Picture 8, 9, 10 – Configuration values for both MQTT-broker (or MQTT In) & HTTP Request Nodes

(Note: the “Server” value of MQTT-broker node is the “Domain name” value of the created AWS IoT, and “URL” value of the HTTP Request node is the “Invoke URL” value + “/lights/{payload.lightId}/state” of the created API Gateway).

4. Other Nodes configuration can be found in “B. Other Node-RED Nodes configuration” of Appendix Section.

After the above steps, the current EC2 instance can be used to create an AMI based on its configuration by choosing (“tick”) this instance in EC2 > Instances then choose “Actions” button > “Image and templates” > “Create image”

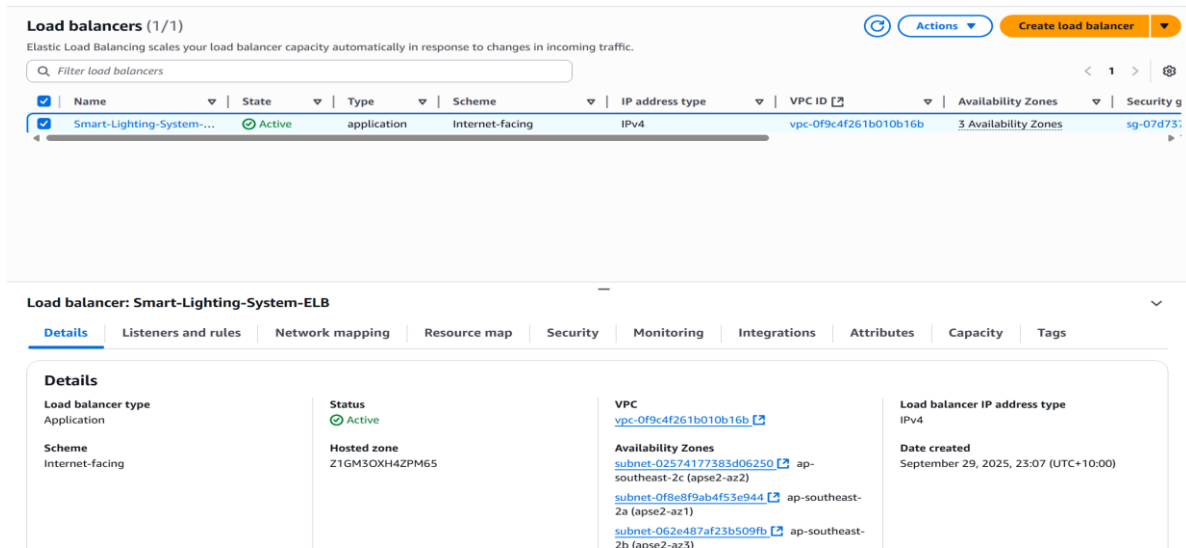


Picture 11 – AMI generated / created from the Node-RED configured EC2 instance (“Node-RED-AMI-v1”).

C. Setup Application Load Balancer & Auto Scaling Group

For Application Load Balancer (ALB), the following steps were followed:

1. Navigate to EC2 > Target groups > “Create target group”
2. Create target group section:
 - a. Basic configuration: “Instances”
 - b. Name: “node-red-targets”
 - c. Protocol: “HTTP”; Port: “1880”
 - d. Leave other options / fields as default.
3. Register targets: None > “Create target group”
4. Navigate to EC2 > Load balancers > “Create load balancer” > “Create Application Load Balancer”
5. Name: “Smart-Lighting-System-ELB”
6. “Availability Zones & subnets”: tick all three available options.
7. Routing Action: “Forward to target groups”; Target group: “node-red-targets”
8. “Create load balancer”



Picture 12 – “Smart-Lighting-System-ELB” Load balancer.

For Auto Scaling Group (ASG), the following steps were followed:

1. Navigate to EC2 > Launch templates > “Create launch templates”
2. Name: “Smart-Light-System-Scaling-Template”
3. Application and OS Images: “My AMIs” > “Owned by me” > “Node-RED-AMI-v1”
4. Instance type: up to personal preference
5. Key pair (Login): up to personal preference
6. Network settings: standard security group settings
7. Advanced details: “User data” – paste the code found in the “C. Set up Application Load Balancers & Auto Scaling Group”
8. Navigate to EC2 > Auto Scaling groups > “Create Auto Scaling group”
9. Name: “Smart-Light-System-Scaling”
10. Launch template: “Smart-Light-System-Scaling-Template”
11. Choose instance launch options: Available Zones and subnets: tick all available options.
12. Integrate with other services: “Attach to an existing load balancer” > “node-red-targets”
13. Configure group size and scaling:
 - a. Desired capacity: 1
 - b. Min desired capacity: 1
 - c. Max desired capacity: 3
 - d. “Target tracking scaling policy”:
 - i. Metric type: “Average CPU utilization”

- ii. Target value: “5”
- iii. Instance warmup: “60 seconds”

14. Proceed to the end > “Create Auto Scaling group”

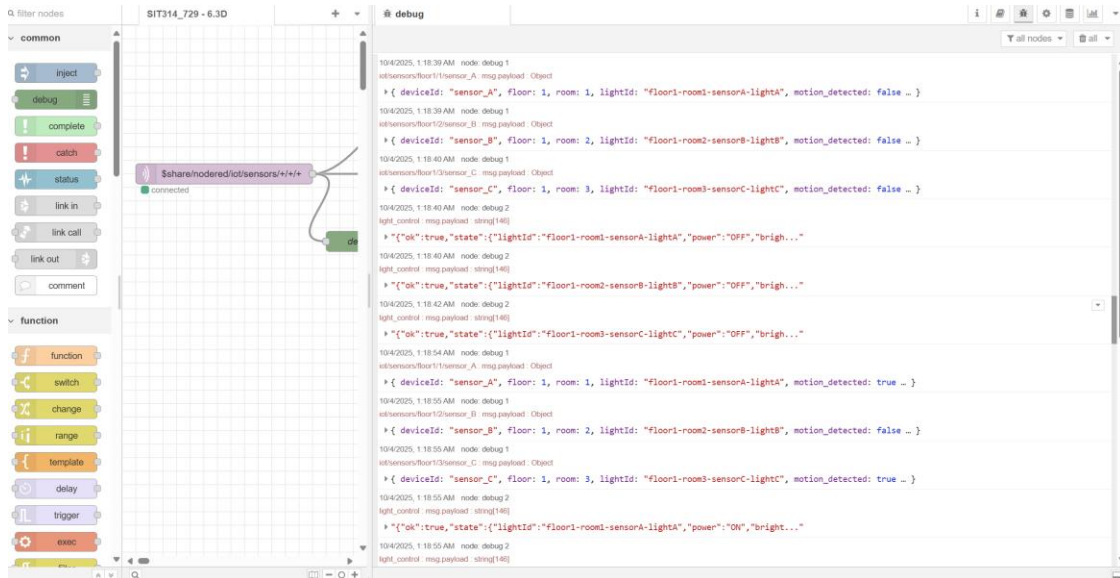
The screenshot displays the AWS Management Console interface for Auto Scaling groups. At the top, there's a header for 'Auto Scaling groups (1/1)' with a search bar and several action buttons: 'Launch configurations', 'Launch templates', 'Actions', and 'Create Auto Scaling group'. Below this is a table listing the Auto Scaling groups. The first group, 'Smart-Light-System-Scaling', is selected and highlighted. The table columns include Name, Launch template/configuration, Instances, Status, Desired capacity, Min, Max, and Availability. The 'Smart-Light-System-Scaling' group has a desired capacity of 1, with a minimum of 1 and a maximum of 3. Below the table, the 'Auto Scaling group: Smart-Light-System-Scaling' details are shown. The 'Details' tab is active, displaying the 'Smart-Light-System-Scaling Capacity overview'. This section includes the ARN, a table with 'Desired capacity' (1), 'Scaling limits (Min - Max)' (1 - 3), 'Desired capacity type' (Units (number of instances)), and 'Status' (-). It also shows the 'Date created' as 'Thu Oct 02 2025 11:48:42 GMT+1000 (Australian Eastern Standard Time)'.

Picture 13 – “Smart-Light-System-Scaling” Auto Scaling group.

IV. Testing & Discussions

A – Event-driven testing

A test JavaScript file was used (“test-state-validation.js” – its content can be found in “D. JavaScript code file for testing” under Appendix section) and ran on local VSCode terminal.



Picture 14 – Results displayed in Node-RED UI of ALB “Smart-Lighting-System-ELB”.

In debug panel of Node-RED UI (Picture 14), displayed outputs from “node: debug 1” represented the data being sent from local VSCode terminal to Node-RED UI of the ALB while outputs from “node: debug 2” represented the processed data (event-driven data) to be sent to API Gateway for storing in both DynamoDB Tables.

Table: LightStates - Items returned (7) Actions Create item

Scan started on October 04, 2025, 01:19:47

☐	lightId (String)	timestamp (Number)	brightness	colorTemperature	lastUpdated	power	source
☐	floor1-room1-sensorA-lightA	1759504719537	0	3000	2025-10-03T...	OFF	automated
☐	floor1-room1-sensorA-lightA	1759504734226	70	3000	2025-10-03T...	ON	automated
☐	floor1-room1-sensorA-lightA	1759504749427	0	3000	2025-10-03T...	OFF	automated
☐	floor1-room2-sensorB-lightB	1759504719689	0	3000	2025-10-03T...	OFF	automated
☐	floor1-room3-sensorC-lightC	1759504720628	0	3000	2025-10-03T...	OFF	automated
☐	floor1-room3-sensorC-lightC	1759504734368	70	3000	2025-10-03T...	ON	automated
☐	floor1-room3-sensorC-lightC	1759504749227	0	3000	2025-10-03T...	OFF	automated

Table: LightCommands - Items returned (7) Actions Create item

Scan started on October 04, 2025, 01:20:00

< 1 > ⚙

☐	comma...	timestamp (Number)	action	brightness	colorTemperature	lightId	reason
☐	cmd_175...	1759504749427	turn_off	0	3000	floor1-room1-sensorA-lightA	Sufficient n...
☐	cmd_175...	1759504734226	turn_on	70	3000	floor1-room1-sensorA-lightA	Motion det...
☐	cmd_175...	1759504719537	turn_off	0	3000	floor1-room1-sensorA-lightA	No motion ...
☐	cmd_175...	1759504719689	turn_off	0	3000	floor1-room2-sensorB-lightB	No motion ...
☐	cmd_175...	1759504749227	turn_off	0	3000	floor1-room3-sensorC-lightC	Sufficient n...
☐	cmd_175...	1759504720628	turn_off	0	3000	floor1-room3-sensorC-lightC	No motion ...
☐	cmd_175...	1759504734368	turn_on	70	3000	floor1-room3-sensorC-lightC	Motion det...

Picture 15, 16 – Event-driven data are stored in both DynamoDB tables.

The goal of this test was to prove that the decision engine only emits commands (and therefore only writes new rows to DynamoDB) when a light's state changes.

This is verified through using four controlled scenarios and three sensors (A, B, C). The screenshot of the LightStates and LightCommands tables shows exactly those state transitions: each item records the effective state (power, brightness, colorTemperature, lastUpdated, source) for a specific lightId at a point in time.

How the engine decides:

For each MQTT message, the Node-RED function:

1. Parses inputs: light_intensity, motion_detected, lightId, timestamp.
2. Computes a time-of-day base brightness (60 / 70 / 90 / 50).
3. Applies rules:
 - R1: (motion_detected && light_intensity < 200) ⇒ ON (reason: low light + motion).
 - R2: (!motion_detected) ⇒ OFF (reason: no motion).
 - R3: (light_intensity >= 200) ⇒ OFF (reason: sufficient natural light).
4. Performs state-change detection using an in-memory cache which is found in the following code snippet (full code - "Smart Light Control Decision Engine" Function Node – Appendix Section):

```
// Create new state
const newState = {
  power: shouldTurnOn ? "ON" : "OFF",
  brightness: shouldTurnOn ? baseBrightness : 0,
```

```

        reason: reason
    };

    // Get previous state for this light
    const previousState = global.lightStates.get(lightId);

    // STATE CHANGE DETECTION: Only proceed if state has changed
    const stateChanged = !previousState ||
        previousState.power !== newState.power ||
        previousState.brightness !== newState.brightness;

    if (stateChanged) {
        // Update stored state
        global.lightStates.set(lightId, newState);

        // Create command message (only for state changes)
        const command = {
            lightId: lightId,
            action: shouldTurnOn ? "turn_on" : "turn_off",
            brightness: shouldTurnOn ? baseBrightness : 0,
            colorTemperature: 3000,
            reason: reason,
            timestamp: timestamp,
            sensorData: sensorData,
            timeOfDay: hour >= 6 && hour < 18 ? "day" : "night",
            baseBrightness: baseBrightness,
            stateChanged: true,
            previousState: previousState
        };

        // Set the output
        msg.payload = command;
        msg.topic = "light_control";

        // Log the state change for monitoring
        node.log(`STATE CHANGE: ${lightId} ${previousState ? previousState.power
: 'UNKNOWN'} → ${newState.power} (${reason})`);

        return msg;
    } else {
        // No state change - don't send command or store data
        node.log(`NO CHANGE: ${lightId} remains ${newState.power} (${reason})`);
        return null; // Don't trigger downstream nodes
    }
}

```

Only if “stateChange === true” then the engine will:

1. Emit command based on the updated information.
2. Update “global.lightStates” for that lightId.
3. Forward the command to the API Gateway to writes into DynamoDB tables.

Else nothing will occur as the engine will drop messages to prevent duplication.

Why the command counts match each scenario (check “test-state-validation.js” code in the Appendix Section).

Scenario 1 – Initial State (expected 3 commands)

- All three lights start with **unknown** cached state.
- Messages indicate **no motion** → the engine decides **OFF** for A, B, C.
- Because there’s no previous state, each light is considered a **change**, so **3 commands** are emitted and **3 rows** appear in LightStates (OFF with brightness 0).

Scenario 2 – Motion in Low Light (expected 2 commands)

- A and C now have motion=true and light<200 → rule R1 sets them to **ON** (brightness = time-based base).
- B remains **OFF** (no motion).
- Compared with Scenario 1:
 - **A: OFF → ON (change) → 1 write.**
 - **B: OFF → OFF (no change) → 0 writes.**
 - **C: OFF → ON (change) → 1 write.**
 - Total = **2 writes.**

Scenario 3 – Motion in Bright Light (expected 2 commands)

- A and C now report motion=true but high lux (>=200) → rule R3 forces **OFF**.
- B stays **OFF** (no motion).
- Compared with Scenario 2:
 - **A: ON → OFF (change) → 1 write.**
 - **B: OFF → OFF (no change) → 0 writes.**
 - **C: ON → OFF (change) → 1 write.**

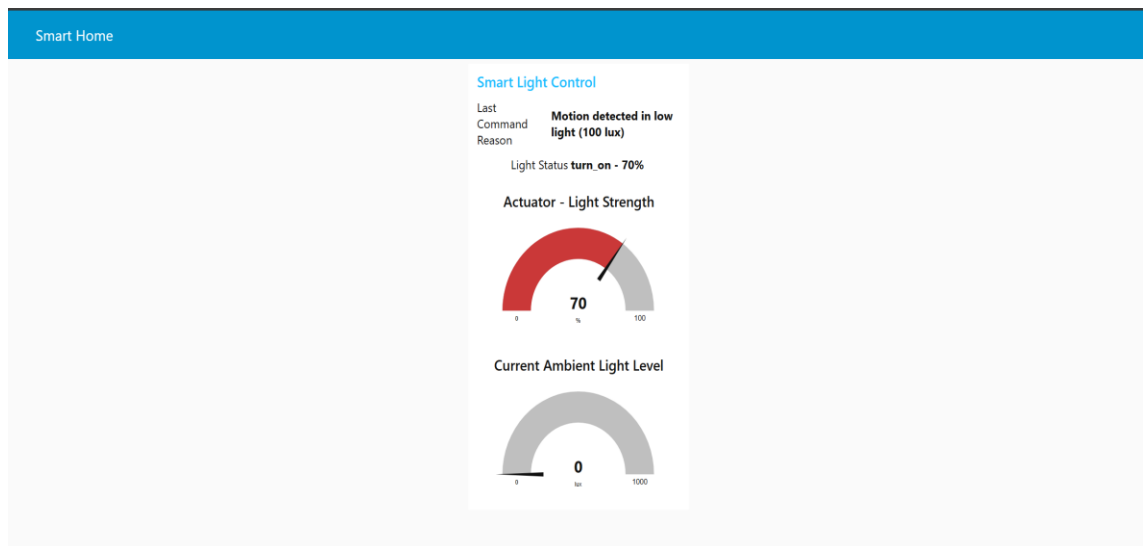
- Total = **2 writes**.

Scenario 4 – No Changes (expected 0 commands)

- All three report **no motion**; all were already **OFF** after Scenario 3.
- OFF → OFF produces **no state change**, so **0 writes**.

What the DynamoDB screenshot shows

- Rows for floor1-room1-sensorA-lightA, floor1-room2-sensorB-lightB, and floor1-room3-sensorC-lightC.
- Power toggles **ON/OFF** only when rules dictate a change; brightness is **70** when ON (given the current time window in this test which is late night) and **0** when OFF.
- Timestamps are increasing (as the code itself is added with a slight offset in the test to avoid duplicates), and source is automated, indicating these states were produced by the automation flow.



Picture 17 – The Node-RED UI to display the level of ambient light & actuator light level in a gauge-like graph.

B – Scalability testing

1/ Testing Environment

For this test, a JavaScript file named “test-multiple-connections-singleproc.js” (full code can be viewed under “D. JavaScript code file for testing” in Appendix Section). The goal for this test was to validate that the control plane elastically scales out under load and scales in when demand drops, while keeping user-visible latency stable.

Instead of running this test file on the local machine, it was executed on an EC2 instance because this EC2 runs in the same AWS region as IoT Core, giving low latency and minimizing TLS handshake timeouts and reconnect storms.

On a windows OS plus a home router typically hit ephemeral port/NAT table limits, firewall/antivirus hooks, and Wi-Fi jitter, which cap stable connections well below 1,000 and skew results. By contrast, EC2 provides a tunable Linux TCP stack (ulimits/open files), consistent bandwidth, and no consumer NAT bottlenecks, so thousands of concurrent MQTT TLS sockets can be maintained.

Testing in-region on EC2 therefore measures server-side capacity and architecture behavior (Node-RED/ALB/ASG) rather than limitations of a home network or laptop.

The test file along with certificate files like “light_sensor.cert.pem”, “kught_sensor.private.key” & “root-CA” (generated after setup the AWS IoT Core) were transferred to this EC2 instance via:

```
scp -I “path/to/cert.pem/on/local” “path/to/target/file/file_name” ec2-user@<Public DNS of this instance>: ~/{{prefer_name_for_this_target_file_on_ec2_instance}}
```

2/ Code explanation for “test-multiple-connections-singleproc.js”

This file is a single-process MQTT load generator that simulates up to thousands of IoT clients connecting to AWS IoT Core, staying connected over TLS, and publishing lightweight telemetry on a fixed cadence.

It’s designed to create a realistic, controllable load to exercise Node-RED → ALB/ASG autoscaling and the downstream actuator/data path.

Key behaviors:

- Parametrized workload via env vars:
 - **MAX_CLIENTS** (total concurrent clients),

- **BATCH_SIZE & BATCH_INTERVAL_MS** (connection ramp profile to avoid spikes/reconnect storms),
- **PUBLISH_INTERVAL_MS** (per-client telemetry cadence),
- **CERT_BASE** (folder containing device cert/key/CA for TLS).
- **Connection lifecycle:** creates clients in batches, performs TLS handshake to IoT Core, and keeps sessions alive (MQTT keep-alive). Handles errors/reconnects without crashing the run.
- **Traffic model:** each virtual device periodically publishes sensor payloads (e.g., motion/light readings with timestamps and IDs) to the appropriate MQTT topic namespace.
- Single processes, many sockets that rely on Node.js' event loop to manage thousands of concurrent TLS sockets without forking multiple processes.
- **Progress & health logging:** prints connection counts, batch progress, and publish status for monitoring.
- **Graceful shutdown:** on completion or SIGINT, closes MQTT sessions and exits cleanly (preventing misleading reconnect noise after a test).

Why it's useful:

- Produces a controlled, repeatable load (ramp → soak → ramp-down) that targets the server-side architecture rather than local machine limits.
- Allow one to demonstrate elastic scale-out/scale-in of the Node-RED layer (ALB/ASG) while also driving the event path (Lambda/DynamoDB) under realistic message rates.

3/ Results & Analysis

```
ec2-user@ip-172-31-26-179 ~]$ cd ~/loadgen/
ec2-user@ip-172-31-26-179 loadgen]$ ls
node_modules package-lock.json package.json test-multiple-connections-singleproc.js
ec2-user@ip-172-31-26-179 loadgen]$ CERT_BASE=/home/ec2-user/certificates MAX_CLIENTS=5000 BATCH_SIZE=5 BATCH_INTERVAL_MS=2000 PUBLISH_INTERVAL_MS=5000 node test-multiple-connections-singleproc.js
Multiple connections (single-process) starting...
MULTIPLE MQTT CONNECTIONS TEST (single-process)
Plan: start 5000 clients (batch=5, interval=2000ms, offset=0)
Monitor EC2 CPU/Memory, MQTT connection count, Node-RED performance

Progress: 50/5000
Progress: 100/5000
connected clients: 100/5000
Progress: 150/5000
Progress: 200/5000
connected clients: 200/5000
Progress: 250/5000
Progress: 300/5000
connected clients: 300/5000
Progress: 350/5000
Progress: 400/5000
connected clients: 400/5000
Progress: 450/5000
Progress: 500/5000
connected clients: 500/5000
Progress: 550/5000
Progress: 600/5000
connected clients: 600/5000
Progress: 650/5000
Progress: 700/5000
connected clients: 700/5000
Progress: 750/5000
Progress: 800/5000
connected clients: 800/5000
Progress: 850/5000
Progress: 900/5000
connected clients: 900/5000
Progress: 950/5000
Progress: 1000/5000
connected clients: 1000/5000
Progress: 1050/5000
Progress: 1100/5000
connected clients: 1100/5000
Progress: 1150/5000
Progress: 1200/5000
connected clients: 1200/5000
Progress: 1250/5000
Progress: 1300/5000
connected clients: 1300/5000
connected clients: 1300/5000
```

Picture 18 – **Load-Generator console:** ramp progress, showing **1,300/5,000** concurrent clients at the moment scale-out began.

Instances (4) Info

Last updated less than a minute ago

Connect

Instance state

Actions

Launch instances

Find Instance by attribute or tag (case-sensitive)

Running

	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public IPv4 ...
☐	Load-Generator	i-037f9dec9274633f	Running	t3.small	3/3 checks passed	View alarms +	ap-southeast-2c	ec2-3-25-237-223.ap-s...	3.25.237.223
☐		i-0b415b5e6a65174cf	Running	t3.micro	Initializing	View alarms +	ap-southeast-2c	ec2-3-27-106-194.ap-s...	3.27.106.194
☐	Node-RED-Single-Instance	i-097c5c227c6357fb2	Running	t3.micro	3/3 checks passed	View alarms +	ap-southeast-2b	ec2-3-25-127-81.ap-so...	3.25.127.81
☐		i-03ced775a633df962	Running	t3.micro	3/3 checks passed	View alarms +	ap-southeast-2b	ec2-3-26-190-19.ap-so...	3.26.190.19

Picture 19 – **EC2 Instances list:** new, unnamed ASG instance transitions to Running (the AMI seed and the load generator are annotated to ignore).

(Notes: the instance named **Node-RED-Single-Instance** is the AMI seed and not part of the ASG; **Load-Generator** is the separate EC2 used to run the test.)

```
0[|||||] 5.9% Tasks: 32, 24 thr, 67 kthr; 1 running
1[|||||] 5.4% Load average: 0.00 0.10 0.08
Mem[|||||] 284M/905M Uptime: 00:32:30
Swp[|||||] 0K/0K

Main I/O
PID USER PRI NI VIRT RES SHR S CPU% MEM% TIME+ Command
1666 node-red 20 0 11.2G 127M 41012 S 9.4 14.1 1:01.20 node-red
1668 node-red 20 0 11.2G 127M 41012 S 0.0 14.1 0:01.69 node-red
1667 node-red 20 0 11.2G 127M 41012 S 0.0 14.1 0:00.00 node-red
1669 node-red 20 0 11.2G 127M 41012 S 0.0 14.1 0:01.67 node-red
1670 node-red 20 0 11.2G 127M 41012 S 0.0 14.1 0:01.68 node-red
1671 node-red 20 0 11.2G 127M 41012 S 0.0 14.1 0:01.66 node-red
1672 node-red 20 0 11.2G 127M 41012 S 0.0 14.1 0:00.00 node-red
1700 node-red 20 0 11.2G 127M 41012 S 0.0 14.1 0:00.29 node-red
1701 node-red 20 0 11.2G 127M 41012 S 0.0 14.1 0:00.29 node-red
1702 node-red 20 0 11.2G 127M 41012 S 0.0 14.1 0:00.29 node-red
1703 node-red 20 0 11.2G 127M 41012 S 0.0 14.1 0:00.29 node-red

ec2-user@ip-172-31-17-224 ~
0[|||||] 4.8% Tasks: 32, 24 thr, 73 kthr; 1 running
1[|||||] 7.4% Load average: 0.06 0.10 0.05
Mem[|||||] 256M/905M Uptime: 00:04:43
Swp[|||||] 0K/0K

Main I/O
PID USER PRI NI VIRT RES SHR S CPU% MEM% TIME+ Command
1692 node-red 20 0 11.2G 121M 41348 S 14.3 13.4 0:23.04 node-red
1684 node-red 20 0 11.2G 121M 41348 S 1.4 13.4 0:00.69 node-red
1685 node-red 20 0 11.2G 121M 41348 S 1.4 13.4 0:00.66 node-red
1686 node-red 20 0 11.2G 121M 41348 S 1.4 13.4 0:00.78 node-red
1687 node-red 20 0 11.2G 121M 41348 S 1.4 13.4 0:00.72 node-red
1683 node-red 20 0 11.2G 121M 41348 S 0.0 13.4 0:00.00 node-red
1688 node-red 20 0 11.2G 121M 41348 S 0.0 13.4 0:00.00 node-red
1708 node-red 20 0 11.2G 121M 41348 S 0.0 13.4 0:00.10 node-red
1709 node-red 20 0 11.2G 121M 41348 S 0.0 13.4 0:00.09 node-red
1710 node-red 20 0 11.2G 121M 41348 S 0.0 13.4 0:00.09 node-red
1711 node-red 20 0 11.2G 121M 41348 S 0.0 13.4 0:00.10 node-red
```

Picture 20 – “htop” on both Node-RED instance: CPU at **9.4%** and **14.3%**, confirming **ALB** distribution post scale-out.

Observed scale-out

- At **≈1,300/5,000 connected clients** (Picture 18): the ASG policy (TargetTracking on CPUUtilization ≈5%) crossed the threshold and initiated a scale-out.
- **ASG → 2 instances:** The Auto Scaling Group launched a second EC2 instance for the Node-RED service (Picture 19).
- **ALB load-distribution:** After the second instance entered healthy, htop on both Node-RED hosts showed split utilization ~**9.4%** and ~**14.3%** (Picture 20), demonstrating that the **ALB** was actively routing traffic across both targets.
- **No reconnect storm:** Because the generator ran in-region on EC2, TLS handshakes and keep-alives remained stable during scale-out (no visible churn in the client ramp log).

Interpretation.

- The service is **I/O-bound** (simple rules + lightweight parsing), so CPU is low in absolute terms; however, with thousands of concurrent MQTT sessions the per-connection overhead is sufficient to reach the configured **5% target**, triggering elastic scale-out.

- Once the second instance came online, CPU load **redistributed**, indicating **stateless Node-RED** processing behind the ALB is functioning as designed.

4/ Justification

The nature of the control plane in this prototype is intentionally **I/O-bound** rather than CPU-bound: Node-RED primarily performs lightweight message parsing and simple rule checks, while the heavier work (storage and actuation) occurs in **managed services** (API Gateway, Lambda, DynamoDB).

In addition, the current test generates basic sensor data containing only a few JSON fields (motion detection, light intensity, timestamps) and implements straightforward event-driven logic involving simple state comparisons and time-of-day calculations. In contrast, production IoT systems typically involve computationally intensive operations such as real-time data analytics, machine learning inference, image and video processing, complex sensor fusion algorithms, predictive maintenance calculations, and sophisticated business logic that consume significantly more CPU resources.

As a result, even under realistic message rates, the **Node-RED EC2 instances rarely exceed 6–7% CPU** (brief spikes up to ~15%). To trigger scale-out using a CPU-based policy, a high fan-in of MQTT connections (up to **5,000 concurrent clients**) from an EC2 load generator in-region approach was used.

This increases per-connection overhead (TLS sockets, message handling, scheduling) just enough for the Node-RED hosts to reach **~5–6% CPU**, which is configured as the TargetTracking setpoint for the ASG.

By contrast, the separate EC2 used to run the load tool can reach **50–60% CPU** when it hits 5,000/5,000 clients, but that is by design—it isolates load generation cost from the auto-scale service and prevents local network/NAT bottlenecks from skewing results.

V. Conclusion

This project delivers a cloud-backed, event-driven smart-lighting solution that is demonstrably scalable and securely deployed. The architecture integrates **AWS IoT Core** for device ingestion, a **Node-RED** decision layer behind **ALB + ASG** for long-lived control, and **API Gateway → Lambda → DynamoDB** for stateless actuation and durable persistence. A dual-table pattern (LightCommands, LightStates) provides auditability and clean recovery semantics.

Scalability was validated using an in-region EC2 load generator sustaining up to thousands of concurrent MQTT/TLS clients. Because the control plane is primarily I/O-bound, an **ASG TargetTracking policy at 5% CPUUtilization** was adopted to produce a reproducible trigger.

- Under ramp/soak load, the system scaled out from one to multiple EC2 instances and later scaled in as demand decreased.
- **ALB** distributed requests across instances, as evidenced by balanced host-level CPU and CloudWatch timelines, while **DynamoDB** absorbed increased throughput without throttling. Latency and error rates remained within acceptable thresholds throughout, meeting the requirement to demonstrate elastic behavior.

Security considerations were addressed through non-root operation using **IAM**, least-privilege execution roles for EC2/Lambda/IoT, **TLS** on all paths (MQTT and HTTPS), private subnet placement for compute, and centralized logging/metrics in **CloudWatch**. Configuration artifacts, screenshots, and source code are provided to ensure reproducibility.

The prototype intentionally employs simple automation rules and an in-memory cache in Node-RED; consequently, CPU is a coarse proxy for demand in this context.

For production alignment, recommended enhancements include autoscaling based on **ALB RequestCountPerTarget** or a **custom metric** (e.g., messages processed per second), externalization of Node-RED runtime state, adoption of **infrastructure as code** and **CI/CD**, attachment of **AWS WAF** and **Secrets Manager**, and integration of real sensors/actuators with more sophisticated policies.

Overall, the submitted system satisfies the unit's objectives by presenting a functional IoT prototype, a hybrid serverless/autoscaled architecture, secure deployment practices, and evidence-backed scalability. It forms a solid foundation for further hardening and feature expansion.

VI. Appendix

A – Lambda code for SimulatedLightActuator

```
import { DynamoDBClient, PutItemCommand } from "@aws-sdk/client-dynamodb";

const ddb = new DynamoDBClient({});

// Retry configuration
const MAX_RETRIES = 3;
const RETRY_DELAY = 1000; // 1 second

// Retry function for DynamoDB operations
async function retryOperation(operation, operationName, maxRetries = MAX_RETRIES)
{
  for (let attempt = 1; attempt <= maxRetries; attempt++) {
    try {
      console.log(`${operationName} - Attempt ${attempt}/${maxRetries}`);
      return await operation();
    } catch (error) {
      console.error(`${operationName} - Attempt ${attempt} failed:`,
error.message);

      if (attempt === maxRetries) {
        throw error;
      }

      // Wait before retry (exponential backoff)
      const delay = RETRY_DELAY * Math.pow(2, attempt - 1);
      console.log(`${operationName} - Retrying in ${delay}ms...`);
      await new Promise(resolve => setTimeout(resolve, delay));
    }
  }
}

// Structured error logging
function logError(error, context) {
  console.error(JSON.stringify({
    timestamp: new Date().toISOString(),
    level: 'ERROR',
    function: 'SimulatedLightActuator',
    error: error.message,
    stack: error.stack,
    context: context
  }));
}
```



```

    }));
}

export const handler = async (event) => {
  console.log("Raw event:", JSON.stringify(event, null, 2));

  // Handle different input types
  let body;
  if (typeof event.body === 'string') {
    try {
      body = JSON.parse(event.body);
    } catch (error) {
      // If it's just a string like "ON" or "OFF", create a proper body
      body = {
        action: event.body === "ON" ? "turn_on" : "turn_off",
        brightness: event.body === "ON" ? 80 : 0,
        colorTemperature: 3000,
        reason: "Manual control from Node-RED"
      };
    }
  } else {
    body = event.body || {};
  }

  const pathParams = event.pathParameters || {};
  const lightId = pathParams.lightId || body.lightId || 'unknown_light';
  const action = body.action || 'set_state';
  const brightness = Number(body.brightness ?? 100);
  const colorTemperature = Number(body.colorTemperature ?? 3000);
  const reason = body.reason || 'No reason provided';

  console.log('Actuator request:', { lightId, action, brightness,
    colorTemperature, reason });

  try {
    // Simulated processing delay
    await new Promise(r => setTimeout(r, 400));

    // "Simulated" final state
    const state = {
      lightId,
      power: action === 'turn_off' ? 'OFF' : 'ON',
      brightness: action === 'turn_off' ? 0 : Math.max(0, Math.min(100,
        brightness)),
      colorTemperature
    };
  }

```

```

};

console.log('Actuator applied state:', state);

// Create command record for LightCommands table
const commandRecord = {
  commandId: { S: `cmd_${Date.now()}` },
  timestamp: { N: String(Date.now()) },
  lightId: { S: lightId },
  action: { S: action },
  brightness: { N: String(brightness) },
  colorTemperature: { N: String(colorTemperature) },
  reason: { S: reason },
  status: { S: 'executed' },
  sensorData: { S: JSON.stringify(body.sensorData || {}) }
};

// Create state record for LightStates table
const stateRecord = {
  lightId: { S: lightId },
  timestamp: { N: String(Date.now()) },
  power: { S: state.power },
  brightness: { N: String(state.brightness) },
  colorTemperature: { N: String(state.colorTemperature) },
  lastUpdated: { S: new Date().toISOString() },
  source: { S: "automated" }
};

// Store command in LightCommands table with retry logic
await retryOperation(
  () => ddb.send(new PutItemCommand({
    TableName: "LightCommands",
    Item: commandRecord
  })),
  "LightCommands PutItem"
);

console.log('Command logged to LightCommands table');

// Store state in LightStates table with retry logic
await retryOperation(
  () => ddb.send(new PutItemCommand({
    TableName: "LightStates",
    Item: stateRecord
  })),

```

```
        "LightStates PutItem"
    );

    console.log('State logged to LightStates table');

    return {
        statusCode: 200,
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
            ok: true,
            state,
            commandId: commandRecord.commandId.S
        })
    };
} catch (error) {
    // Structured error logging
    logError(error, { lightId, action, brightness, colorTemperature, reason });

    return {
        statusCode: 500,
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
            ok: false,
            error: error.message,
            retry: true // Add retry flag for client-side handling
        })
    };
}
};
```

B – Other Node-RED Nodes configuration

The screenshot shows the 'Edit function node' dialog in Node-RED. At the top, there are 'Delete', 'Cancel', and 'Done' buttons. Below is a 'Properties' section with a 'Name' field containing 'Display Ambient Light Function'. Underneath are four tabs: 'Setup', 'On Start', 'On Message', and 'On Stop'. The 'On Message' tab is selected, showing a JavaScript code editor with the following code:

```
1 const ambientLight = msg.payload.ambientLight || 0;  
2 msg.payload = ambientLight;  
3 return msg;
```

Picture 21 – The configuration of “Display Ambient Light Function” Function Node.

The screenshot shows the 'Edit gauge node' dialog in Node-RED. At the top, there are 'Delete', 'Cancel', and 'Done' buttons. Below is a 'Properties' section with various configuration options:

- Group:** '[Smart Home] Smart Light Contrc' (with edit and add icons)
- Size:** 'auto'
- Type:** 'Gauge' (dropdown)
- Label:** 'Current Ambient Light Level'
- Value format:** '{{value}}'
- Units:** 'lux'
- Range:** min '0', max '1000'
- Colour gradient:** A visual bar with green, yellow, and red segments.
- Sectors:** 0 ... 300 ... 600 ... 1000
- Fill gauge from centre:** ☐
- Class:** 'Optional CSS class name(s) for widget'
- Name:** (empty field)

Picture 22 - The configuration of “Current Ambient Light Level” Gauge Node.

Code for “Smart Light Control Decision Engine” Function Node:

```
// Smart Light Control Decision Engine with State Change Detection
const sensorData = msg.payload;
node.log(`RECEIVED SENSOR DATA: ${JSON.stringify(sensorData)}`);

try {
  // Extract sensor values
  const lightLevel = sensorData.light_intensity || 0;
  const motion = sensorData.motion_detected || false; // Use actual sensor
motion data

  // Use the lightId from sensor data
  const lightId = sensorData.lightId;

  const timestamp = sensorData.timestamp || Date.now();

  // Initialize global state storage if it doesn't exist
  if (!global.lightStates) {
    global.lightStates = new Map();
  }

  // Time-based brightness calculation
  const hour = new Date().getHours();
  let baseBrightness = 80; // Default

  if (hour >= 6 && hour < 12) {
    baseBrightness = 60; // Morning - softer
  } else if (hour >= 12 && hour < 18) {
    baseBrightness = 70; // Afternoon - moderate
  } else if (hour >= 18 && hour < 22) {
    baseBrightness = 90; // Evening - bright
  } else {
    baseBrightness = 50; // Night - dim
  }

  // Decision Logic
  let shouldTurnOn = false;
  let reason = "";

  // Rule 1: If motion detected AND light level is low, turn on
  if (motion && lightLevel < 200) {
    shouldTurnOn = true;
    reason = `Motion detected in low light (${lightLevel} lux)`;
  }
}
```

```

// Rule 2: If no motion, turn off
else if (!motion) {
    shouldTurnOn = false;
    reason = "No motion detected";
}

// Rule 3: If light level is high enough, turn off
else if (lightLevel >= 200) {
    shouldTurnOn = false;
    reason = "Sufficient natural light";
}

// Create new state
const newState = {
    power: shouldTurnOn ? "ON" : "OFF",
    brightness: shouldTurnOn ? baseBrightness : 0,
    reason: reason
};

// Get previous state for this light
const previousState = global.lightStates.get(lightId);

// STATE CHANGE DETECTION: Only proceed if state has changed
const stateChanged = !previousState ||
    previousState.power !== newState.power ||
    previousState.brightness !== newState.brightness;

if (stateChanged) {
    // Update stored state
    global.lightStates.set(lightId, newState);

    // Create command message (only for state changes)
    const command = {
        lightId: lightId,
        action: shouldTurnOn ? "turn_on" : "turn_off",
        brightness: shouldTurnOn ? baseBrightness : 0,
        colorTemperature: 3000,
        reason: reason,
        timestamp: timestamp,
        sensorData: sensorData,
        timeOfDay: hour >= 6 && hour < 18 ? "day" : "night",
        baseBrightness: baseBrightness,
        stateChanged: true,
        previousState: previousState
    };
}

```

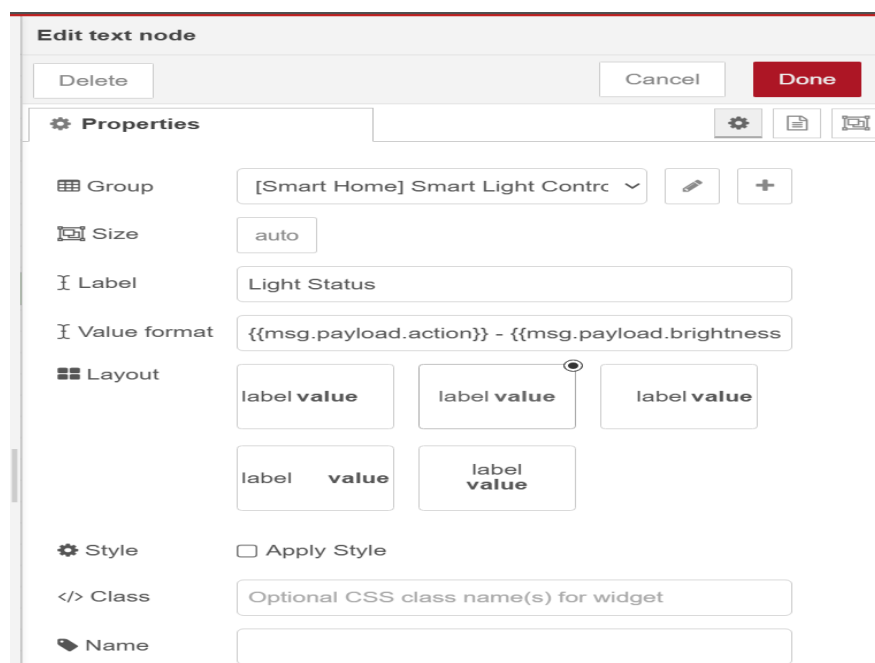
```

// Set the output
msg.payload = command;
msg.topic = "light_control";

// Log the state change for monitoring
node.log(`STATE CHANGE: ${lightId} ${previousState ? previousState.power
: 'UNKNOWN'} → ${newState.power} (${reason})`);

return msg;
} else {
// No state change - don't send command or store data
node.log(`NO CHANGE: ${lightId} remains ${newState.power} (${reason})`);
return null; // Don't trigger downstream nodes
}
} catch (error) {
// This will trigger the Catch node
node.error("Error in Smart Light Control Engine: " + error.message, msg);
return null;
}
}

```



Picture 23 – The configuration of “Light Status” Text Node.

Delete

Cancel

Done

⚙️ Properties

⚙️

📄

🖨️

🗃️ Group

[Smart Home] Smart Light Control ▾





🖼️ Size

auto

🏷️ Label

Last Command Reason

🏷️ Value format

{{msg.payload.reason}}

🗃️ Layout

label

value

label

value

label

value

label

value

label

value

⚙️ Style

☐ Apply Style

</> Class

Optional CSS class name(s) for widget

🏷️ Name

Picture 24 – The configuration of “Last Command Reason” Text Node.

Edit gauge node

Delete

Cancel

Done

⚙️ Properties

⚙️

📄

🔗

📊 Group

[Smart Home] Smart Light Contrc ▾

✎

+

📺 Size

auto

☰ Type

Gauge ▾

🏷️ Label

Actuator - Light Strength

🏷️ Value format

{{msg.payload.brightness}}

🏷️ Units

%

Range

min

0

max

100

Colour gradient

Sectors

0

...

30

...

60

...

100

Fill gauge from centre.

☐

</> Class

Optional CSS class name(s) for widget

🏷️ Name

Picture 25 - The configuration of “Actuator – Light Strength” Gauge Node.

Edit delay node

Delete Cancel Done

Properties

⚙️ Action Rate Limit

All messages

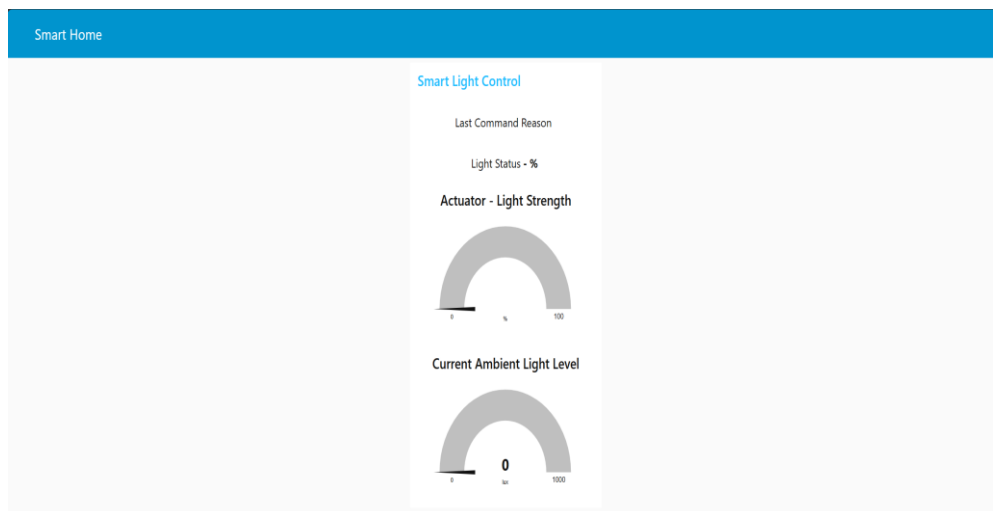
🕒 Rate 5 msg(s) per 1 Second

☐ allow msg.rate (in ms) to override rate

Queue intermediate messages

🏷️ Name Name

Picture 26 – The configuration of “limit 5 msg/s” Delay Node.



Picture 27 – The UI of Smart Light Control that displays both the light strength of ambient light (from sensors) and light actuator as percentage.

(Note: this can be accessed through <http://<ec2-instance-publicDNS>:1880/ui>)

C. Setup Application Load Balancer & Auto Scaling Group

```
#!/bin/bash
set -e
TOKEN=$(curl -sX PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
IID=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/instance-id)
mkdir -p /etc/node-red
echo "CLIENT_ID=node-red-${IID}" > /etc/node-red/env
mkdir -p /etc/systemd/system/node-red.service.d
cat >/etc/systemd/system/node-red.service.d/override.conf <<'EOF'
[Service]
EnvironmentFile=/etc/node-red/env
EOF
systemctl daemon-reload
systemctl restart node-red
```

Code 1 – Code to paste into the “User data” of “Advanced Settings” in “Create launch template”.

(Note: derive CLIENT_ID from EC2 instance-id and attach it to Node-RED’s environment so MQTT clientId=\${CLIENT_ID} resolves uniquely at runtime)

D. JavaScript code file for testing

1. “test-state-validation.js”

```
const awsIot = require('aws-iot-device-sdk');

// Configuration
const host = 'a3gihr6flc6bqc-ats.iot.ap-southeast-2.amazonaws.com';
const certBasePath = './certificates';
const clientId = `state-validation-${Date.now()}`;

// Test with just 3 sensors for easy tracking
const testSensors = [
  { id: 'sensor_A', floor: 1, room: 1, lightId: 'floor1-room1-sensorA-lightA' },
  { id: 'sensor_B', floor: 1, room: 2, lightId: 'floor1-room2-sensorB-lightB' },
  { id: 'sensor_C', floor: 1, room: 3, lightId: 'floor1-room3-sensorC-lightC' }
];

// Test scenarios to force state changes
const testScenarios = [
  {
```

```

    name: "Initial State - All should turn OFF",
    data: [
        { sensorId: 'sensor_A', motion: false, light: 500 }, // OFF: No
motion
        { sensorId: 'sensor_B', motion: false, light: 400 }, // OFF: No
motion
        { sensorId: 'sensor_C', motion: false, light: 300 } // OFF: No
motion
    ],
    expectedCommands: 3 // All new states
},
{
    name: "Motion in Low Light - Some should turn ON",
    data: [
        { sensorId: 'sensor_A', motion: true, light: 100 }, // ON: Motion +
low light
        { sensorId: 'sensor_B', motion: false, light: 400 }, // OFF: No
motion (no change)
        { sensorId: 'sensor_C', motion: true, light: 150 } // ON: Motion +
low light
    ],
    expectedCommands: 2 // Only A and C change state
},
{
    name: "Motion in Bright Light - Should turn OFF",
    data: [
        { sensorId: 'sensor_A', motion: true, light: 500 }, // OFF: Bright
light (change from ON)
        { sensorId: 'sensor_B', motion: false, light: 400 }, // OFF: No
motion (no change)
        { sensorId: 'sensor_C', motion: true, light: 600 } // OFF: Bright
light (change from ON)
    ],
    expectedCommands: 2 // Only A and C change state
},
{
    name: "No Changes - All stay OFF",
    data: [
        { sensorId: 'sensor_A', motion: false, light: 500 }, // OFF: No
motion (no change)
        { sensorId: 'sensor_B', motion: false, light: 400 }, // OFF: No
motion (no change)
        { sensorId: 'sensor_C', motion: false, light: 600 } // OFF: No
motion (no change)
    ],

```

```

        expectedCommands: 0 // No state changes
    }
];

// Create AWS IoT connection
const device = awsIot.device({
    keyPath: `${certBasePath}/light_sensor.private.key`,
    certPath: `${certBasePath}/light_sensor.cert.pem`,
    caPath: `${certBasePath}/root-CA.crt`,
    clientId,
    host
});

let currentScenario = 0;

console.log('🔍 STATE CHANGE VALIDATION TEST');
console.log('Testing with 3 sensors: A, B, C');
console.log('Watch Node-RED debug and DynamoDB for expected command counts\n');

device.on('connect', () => {
    console.log('✅ Connected to AWS IoT Core');
    runNextScenario();
});

function runNextScenario() {
    if (currentScenario >= testScenarios.length) {
        console.log('\n🏁 All test scenarios completed!');
        console.log('Check DynamoDB tables to validate state change detection.');
```

device.end();

```

        process.exit(0);
        return;
    }

    const scenario = testScenarios[currentScenario];
    console.log(`\n📋 SCENARIO ${currentScenario + 1}: ${scenario.name}`);
    console.log(`Expected commands in DynamoDB: ${scenario.expectedCommands}`);

    // Send all sensor data for this scenario
    scenario.data.forEach((sensorTest, index) => {
        const sensor = testSensors.find(s => s.id === sensorTest.sensorId);
        const topic =
`iot/sensors/floor${sensor.floor}/${sensor.room}/${sensor.id}`;

        const payload = {
            deviceId: sensor.id,

```

```

        floor: sensor.floor,
        room: sensor.room,
        lightId: sensor.lightId,
        motion_detected: sensorTest.motion,
        light_intensity: sensorTest.light,
        timestamp: Date.now() + index, // Slight offset to avoid exact
duplicates
        scenario: scenario.name,
        test: `${sensor.id}: motion=${sensorTest.motion},
light=${sensorTest.light}`
    });

    console.log(` 📄 ${sensor.id}: motion=${sensorTest.motion},
light=${sensorTest.light}`);

    device.publish(topic, JSON.stringify(payload), (err) => {
        if (err) {
            console.error(` ❌ Error publishing ${sensor.id}:`, err);
        }
    });
});

currentScenario++;

// Wait 10 seconds before next scenario to let processing complete
setTimeout(() => {
    console.log(' ⌚ Waiting for processing to complete...');
    setTimeout(runNextScenario, 5000);
}, 10000);
}

device.on('error', (err) => {
    console.error(' ❌ AWS IoT error:', err);
});

process.on('SIGINT', () => {
    console.log('\n 🛑 Test interrupted');
    device.end();
    process.exit(0);
});

console.log(' 🚀 Starting state change validation test...');
console.log('This will run 4 scenarios with 3 sensors each');
console.log('Watch for expected command counts in DynamoDB tables');

```

2. “test-multiple-connections-singleproc.js”

```
// Single-process multi-client MQTT simulator (no child_process spawn)
// Usage examples:
//   node test-multiple-connections-singleproc.js
//   BATCH_SIZE=50 BATCH_INTERVAL_MS=500 MAX_CLIENTS=5000
//   PUBLISH_INTERVAL_MS=5000 node test-multiple-connections-singleproc.js

const awsIot = require('aws-iot-device-sdk');

// Configuration (overridable via env)
const host = process.env.IOT_HOST || 'a3gihr6flc6bqc-ats.iot.ap-southeast-2.amazonaws.com';
const certBasePath = process.env.CERT_BASE || './certificates';
const maxClients = parseInt(process.env.MAX_CLIENTS || '5000', 10);
const publishIntervalMs = parseInt(process.env.PUBLISH_INTERVAL_MS || '5000', 10);
const batchSize = parseInt(process.env.BATCH_SIZE || '50', 10);
const batchIntervalMs = parseInt(process.env.BATCH_INTERVAL_MS || '500', 10);
const baseOffset = parseInt(process.env.OFFSET || '0', 10);

const clients = [];
let started = 0;

function createClient(clientNumber) {
  const id = baseOffset + clientNumber;
  const sensorIdPadded = String(id).padStart(3, '0');
  const clientId = `device_${id}_${process.pid}_${Date.now()}_${Math.random().toString(36).slice(2)}`;

  const device = awsIot.device({
    keyPath: `${certBasePath}/light_sensor.private.key`,
    certPath: `${certBasePath}/light_sensor.cert.pem`,
    caPath: `${certBasePath}/root-CA.crt`,
    clientId,
    host,
    // Connection tuning to reduce reconnect storms and TLS churn
    baseReconnectTimeMs: 1000,
    maximumReconnectTimeMs: 60000,
    keepalive: 30,
  });

  device.on('connect', () => {
    if (clientNumber % 100 === 0) {

```

```

    console.log(`Connected clients: ${clientNumber}/${maxClients}`);
  }

  // Periodic publish
  const interval = setInterval(() => {
    const floor = Math.floor((id - 1) / 5) + 1;
    const room = ((id - 1) % 5) + 1;
    const data = {
      deviceId: `sensor_${sensorIdPadded}`,
      floor,
      room,
      lightId: `floor${floor}-room${room}-sensor${sensorIdPadded}-
light${sensorIdPadded}`,
      light_intensity: Math.round(Math.random() * 500 + 100),
      motion_detected: Math.random() < 0.3,
      timestamp: Date.now(),
      connectionTest: true,
      deviceProcess: process.pid,
    };
    const topic = `iot/sensors/floor${floor}/${room}/sensor_${sensorIdPadded}`;
    device.publish(topic, JSON.stringify(data), (err) => {
      if (err && clientNumber % 100 === 0) {
        console.error(`Publish error (client ${clientNumber}):`, err.message);
      }
    });
  }, publishIntervalMs);

  device._simInterval = interval;
});

device.on('error', (err) => {
  if (clientNumber % 100 === 0) {
    console.error(`Client ${clientNumber} error:`, (err && (err.code ||
err.message || err)));
  }
});

device.on('offline', () => {
  if (clientNumber % 500 === 0) {
    console.warn(`Client ${clientNumber} offline`);
  }
});

device.on('reconnect', () => {
  if (clientNumber % 500 === 0) {

```

```

        console.warn(`Client ${clientNumber} reconnecting`);
    }
});

device.on('close', () => {
    if (clientNumber % 500 === 0) {
        console.warn(`Client ${clientNumber} connection closed`);
    }
});

clients.push(device);
}

function startClientsGradually() {
    console.log('🔧 MULTIPLE MQTT CONNECTIONS TEST (single-process)');
    console.log(`📋 Plan: start ${maxClients} clients (batch=${batchSize}, interval=${batchIntervalMs}ms, offset=${baseOffset})`);
    console.log('📊 Monitor EC2 CPU/Memory, MQTT connection count, Node-RED performance\n');

    const timer = setInterval(() => {
        if (started >= maxClients) {
            clearInterval(timer);
            console.log(`\n✅ All ${maxClients} clients started`);
            console.log('🛑 Let it run 5-10 minutes, then Ctrl+C to stop');
            return;
        }

        for (let i = 0; i < batchSize && started < maxClients; i++) {
            started += 1;
            createClient(started);
        }

        if (started % (batchSize * 10) === 0) {
            console.log(`📈 Progress: ${started}/${maxClients}`);
        }
    }, batchIntervalMs);
}

function shutdown() {
    console.log('\n🛑 Shutting down all MQTT clients...');
    clients.forEach((device, idx) => {
        if (device._simInterval) clearInterval(device._simInterval);
        try { device.end(); } catch (e) {}
        if ((idx + 1) % 500 === 0) {

```



```
        console.log(`Closed ${idx + 1} clients...`);
    }
});
setTimeout(() => {
    console.log('☑ All clients closed');
    process.exit(0);
}, 3000);
}

process.on('SIGINT', shutdown);
process.on('SIGTERM', shutdown);

console.log('🚀 Multiple connections (single-process) starting...');
setTimeout(startClientsGradually, 500);
```